```
import random
import time
import numpy as np

def partition(arr, low, high, pivot_index):
    # pick the pivot
    pivot = arr[pivot_index]

    # make a temporary array
    temp = [0] * (high - low + 1)

    # start an index
    idx = 0

    # put elements smaller than pivot into temp
    for i in range(low, high + 1):
        if arr[i] < pivot:
            temp[idx] = arr[i]
            idx += 1
    # place the pivot into temp
    pivot_final_index = idx
    temp[idx] = pivot
    idx += 1

    # put elements greater than or equal to pivot into temp
    for i in range(low, high + 1):
        if i != pivot_index and arr[i] >= pivot:
            temp[idx] = arr[i]
            idx += 1

    # copy temp back into arr
    for i in range(len(temp)):
        arr[low + i] = temp[i]

    return low + pivot_final_index

def partitionInPlace(arr, low, high, pivot_index):
    # move the pivot to the end
    pivot = arr[pivot_index]
    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]

    # start the position for smaller elements
    idx = low

    # move elements smaller than pivot to the left
    for i in range(low, high):
        if arr[i] < pivot:
            arr[idx], arr[i] = arr[i], arr[idx]
            idx += 1

    # put pivot in its final position
    arr[idx], arr[high] = arr[high], arr[idx]

    return idx


def randomizedSelectNewArray(arr, low, high, k):
    # base Case
    if low == high:
        return arr[low]

    # pick a random pivot index
    pivot_index = random.randint(low, high)

    # partition the array
    pivot_index = partition(arr, low, high, pivot_index)

    # find the pivot's rank
    rank = pivot_index - low

    # compare k with rank
    if k == rank:
        return arr[pivot_index]

    elif k < rank:
```

```
        return randomizedSelectNewArray(arr, low, pivot_index - 1, k)

    else:
        return randomizedSelectNewArray(arr, pivot_index + 1, high, k - rank - 1)

def randomizedSelectInPlace(arr, low, high, k):
    # base Case
    if low == high:
        return arr[low]

    # pick a random pivot index
    pivot_index = random.randint(low, high)

    # partition the array
    pivot_index = partitionInPlace(arr, low, high, pivot_index)

    # find the pivot's rank
    rank = pivot_index - low

    # compare k with rank
    if k == rank:
        return arr[pivot_index]

    elif k < rank:
        return randomizedSelectInPlace(arr, low, pivot_index - 1, k)

    else:
        return randomizedSelectInPlace(arr, pivot_index + 1, high, k - rank - 1)

# example
arr = [7, 2, 9, 4, 1, 6, 3]

k = 4

result = randomizedSelectNewArray(arr, 0, len(arr) - 1, k)

print("New array: Value of the ", k,"th smallest element:", result)

# ----------------------------
# Experiment
# ----------------------------
# Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]
# 1. Time RSelect
arr_copy = arr[:]
start = time.perf_counter()
result_rselect = randomizedSelectNewArray(arr_copy, 0, len(arr_copy)-1, k)
t_rselect = time.perf_counter() - start
print(f"NewArrayRSelect time: {t_rselect:.6f} sec")
# 2. Verify correctness against numpy sort
# Time Numpy sort
arr_copy = arr[:]
start = time.perf_counter()
np_sorted = np.sort(arr_copy)
t_numpy = time.perf_counter() - start
print(f"NumPy Sort time: {t_numpy:.6f} sec")

# ----------------------------
# Experiment
# ----------------------------
# Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]
# 1. Time RSelect
arr_copy = arr[:]
start = time.perf_counter()
result_rselect = randomizedSelectInPlace(arr_copy, 0, len(arr_copy)-1, k)
t_rselect = time.perf_counter() - start
print(f"InPlaceRSelect time: {t_rselect:.6f} sec")
# 2. Verify correctness against numpy sort
# Time Numpy sort
arr_copy = arr[:]
start = time.perf_counter()
np_sorted = np.sort(arr_copy)
t_numpy = time.perf_counter() - start
print(f"NumPy Sort time: {t_numpy:.6f} sec")
```

```
New array: Value of the  4 th smallest element: 6
NewArrayRSelect time: 0.005199 sec
NumPy Sort time: 0.004023 sec
InPlaceRSelect time: 0.000805 sec
NumPy Sort time: 0.000475 sec
```